



Kubernetes Fundamental



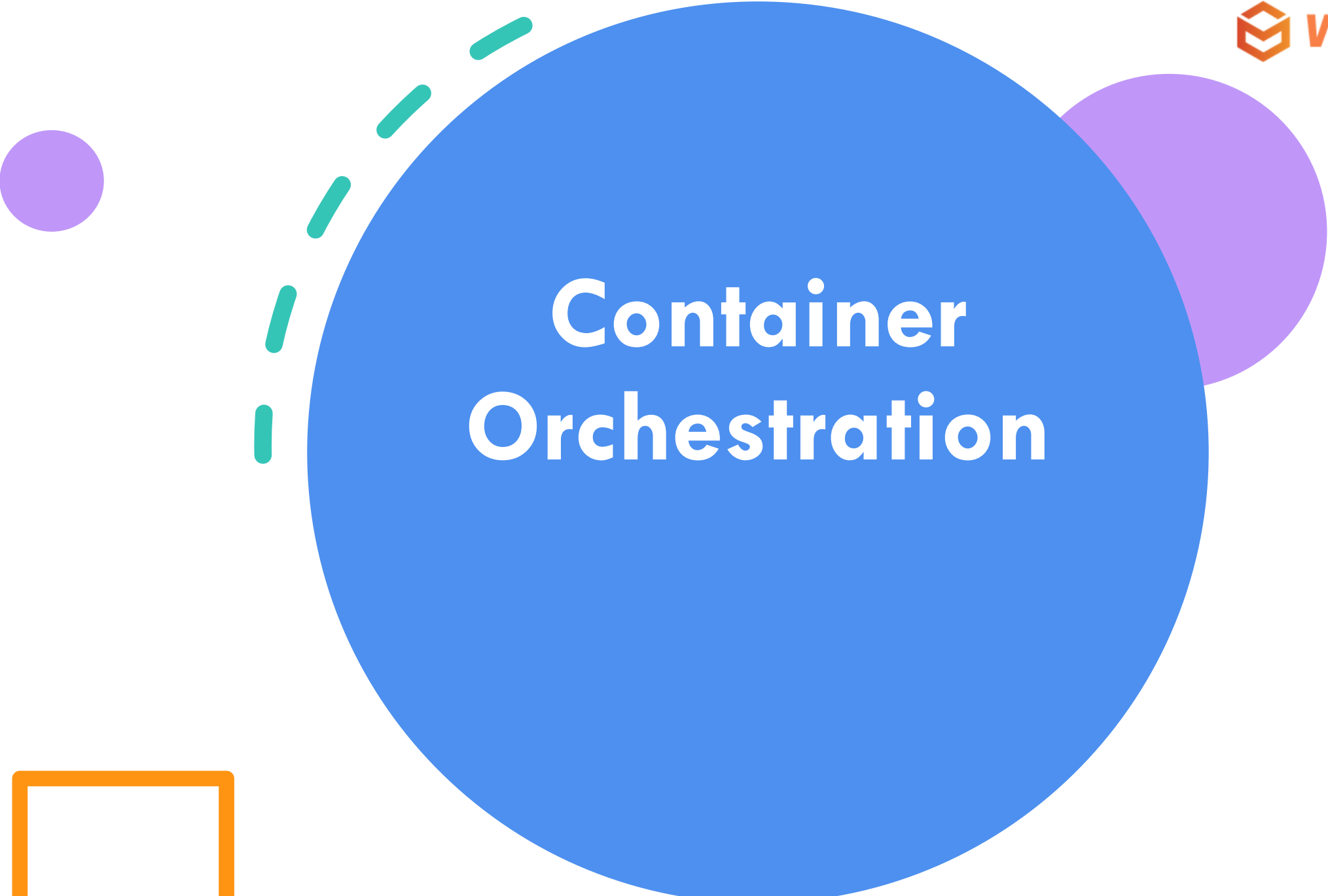
Today's Lesson

Container Orchestration & Kubernetes
Architecture

K8s basic object: Pods, Services,
Deployments, ConfigMaps, Secrets

Service types and communication

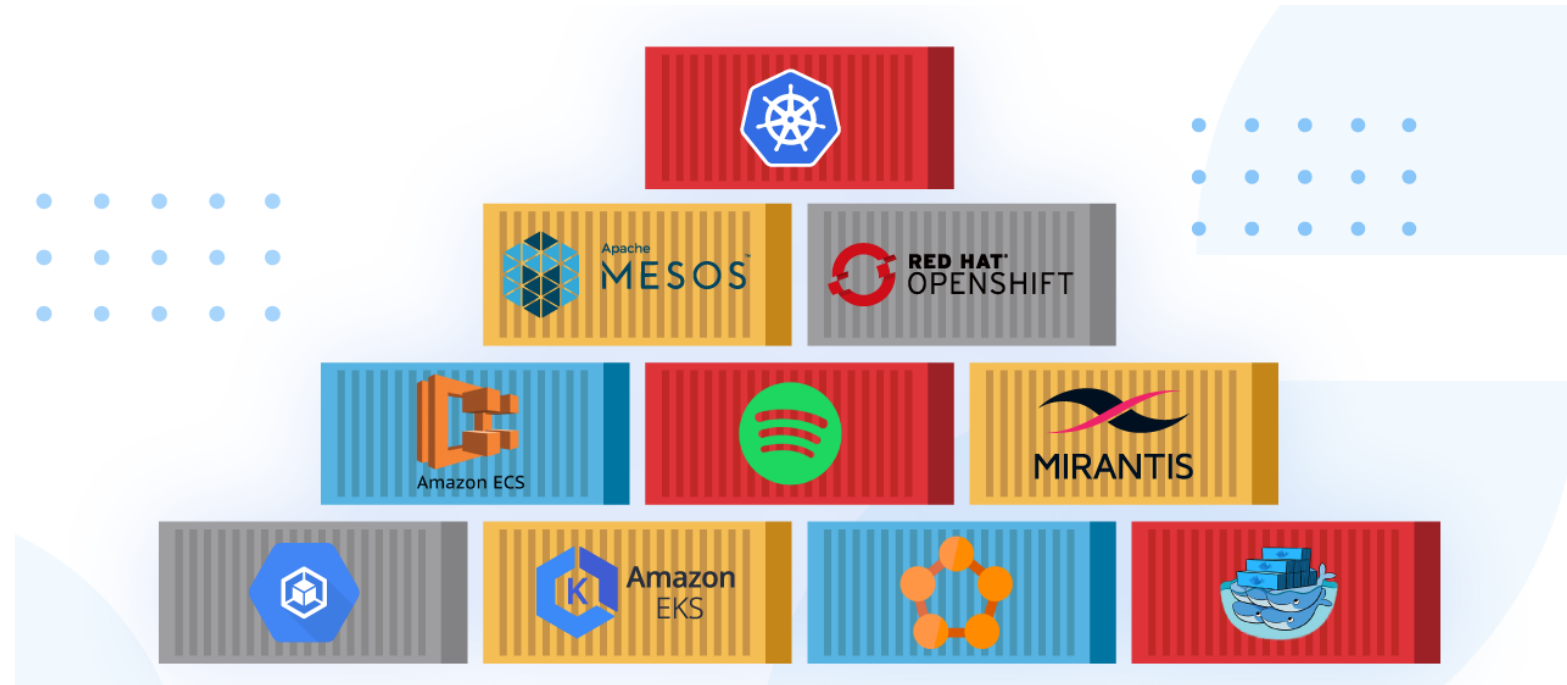
Command-line tool mastery



Container Orchestration

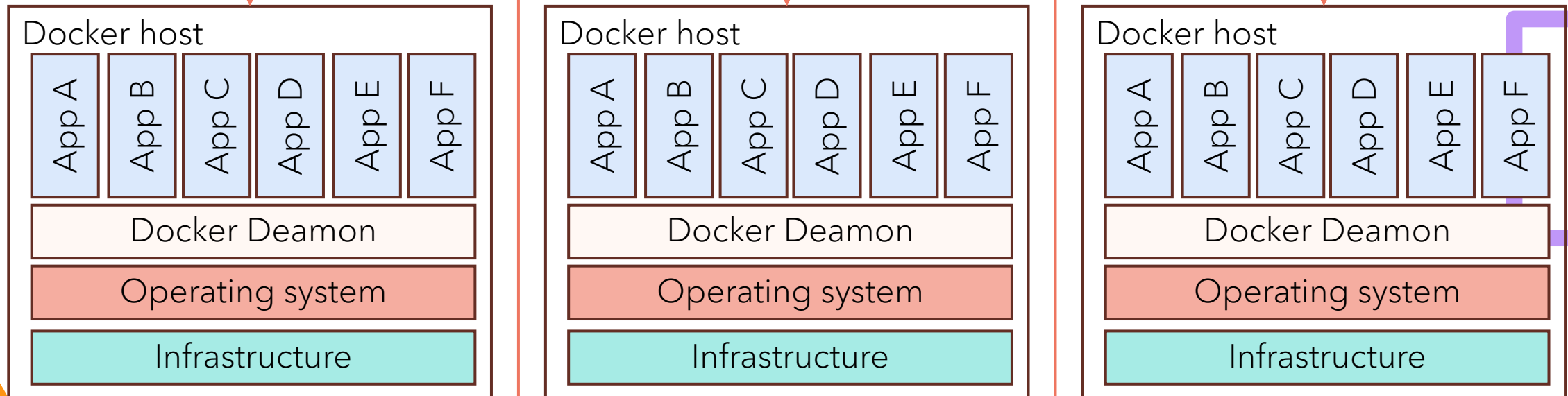
Container Orchestration

- Container orchestration is the automated management of the entire lifecycle of containerized applications, handling tasks like deployment, scaling, networking, and resource allocation across **multiple hosts**.



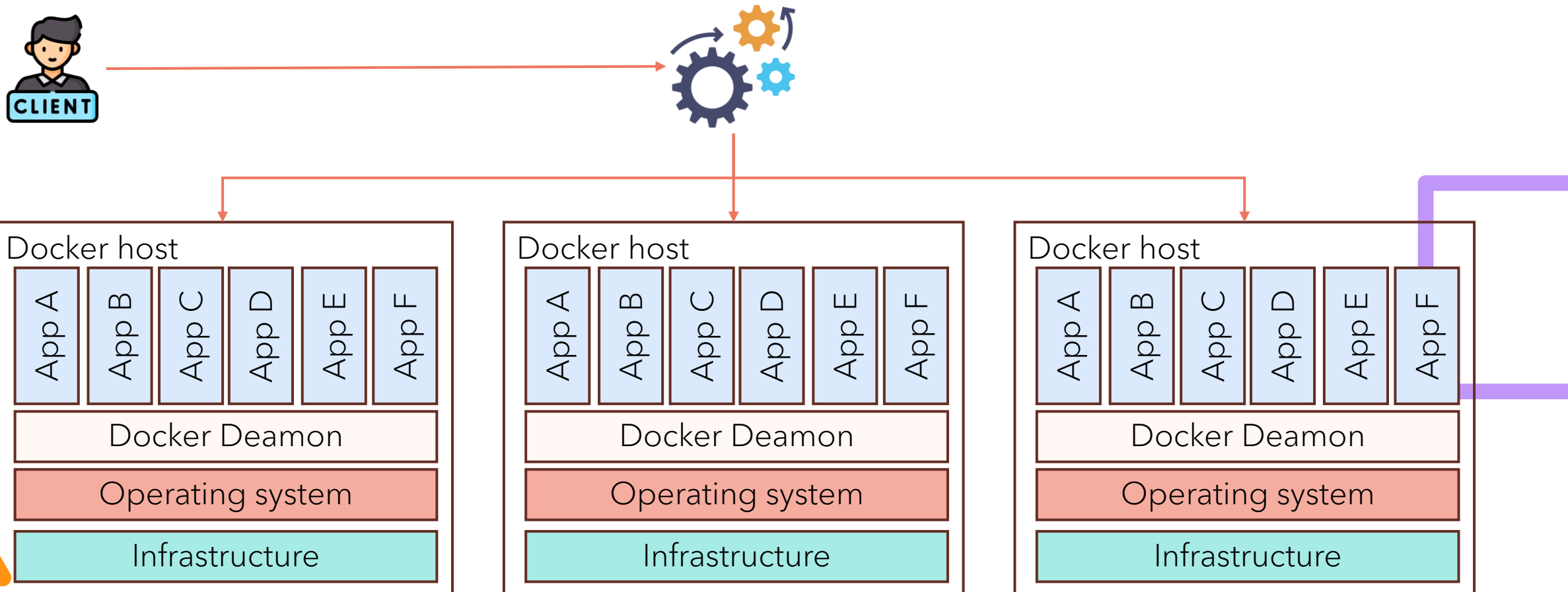
Container Orchestration

- With docker, we managed each Container Host (VM) separately



Container Orchestration

- With Orchestration, we managed Container Host (VM) by using Orchestration Tool (Controller)

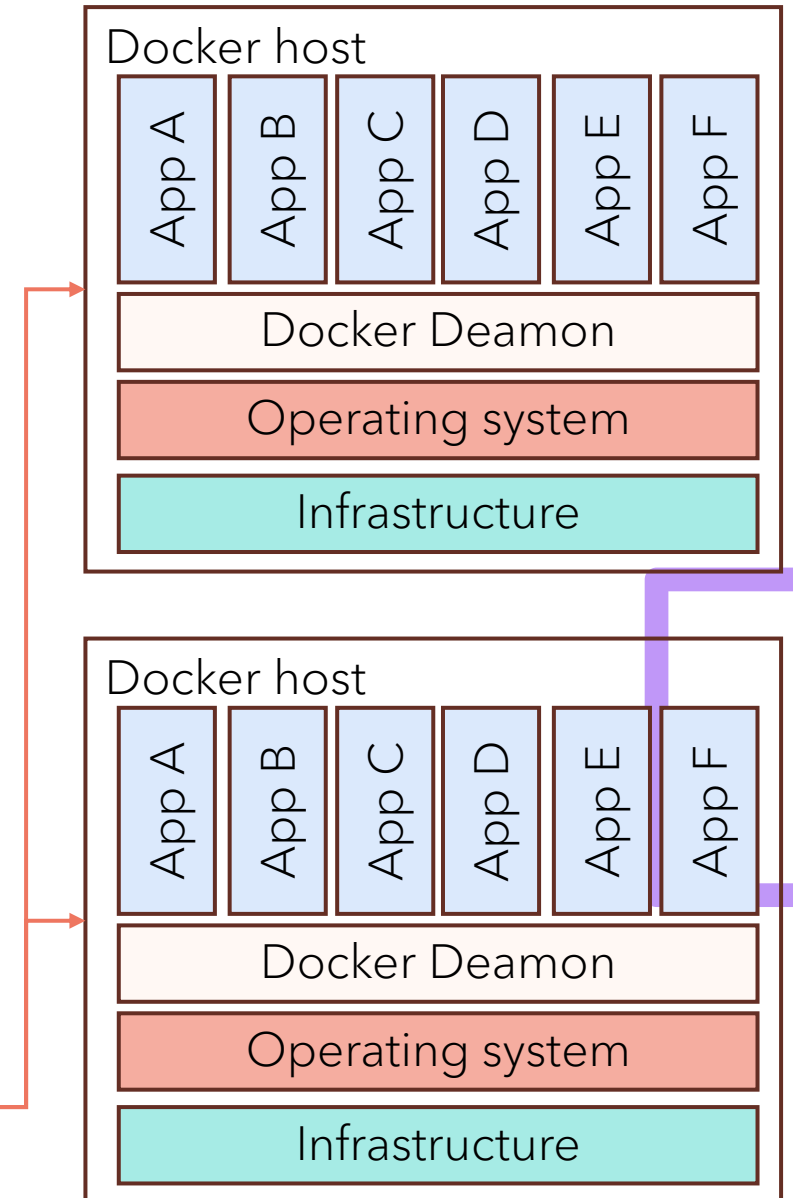
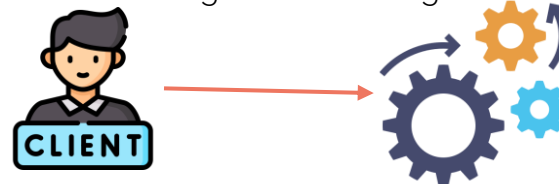


Container Orchestration

When you run just a few containers (for example, using **Docker**), you can easily manage them manually. But in real-world scenarios, organizations often deal with **hundreds or thousands of containers** that need to run across **multiple servers**.

Key Functions of Container Orchestration

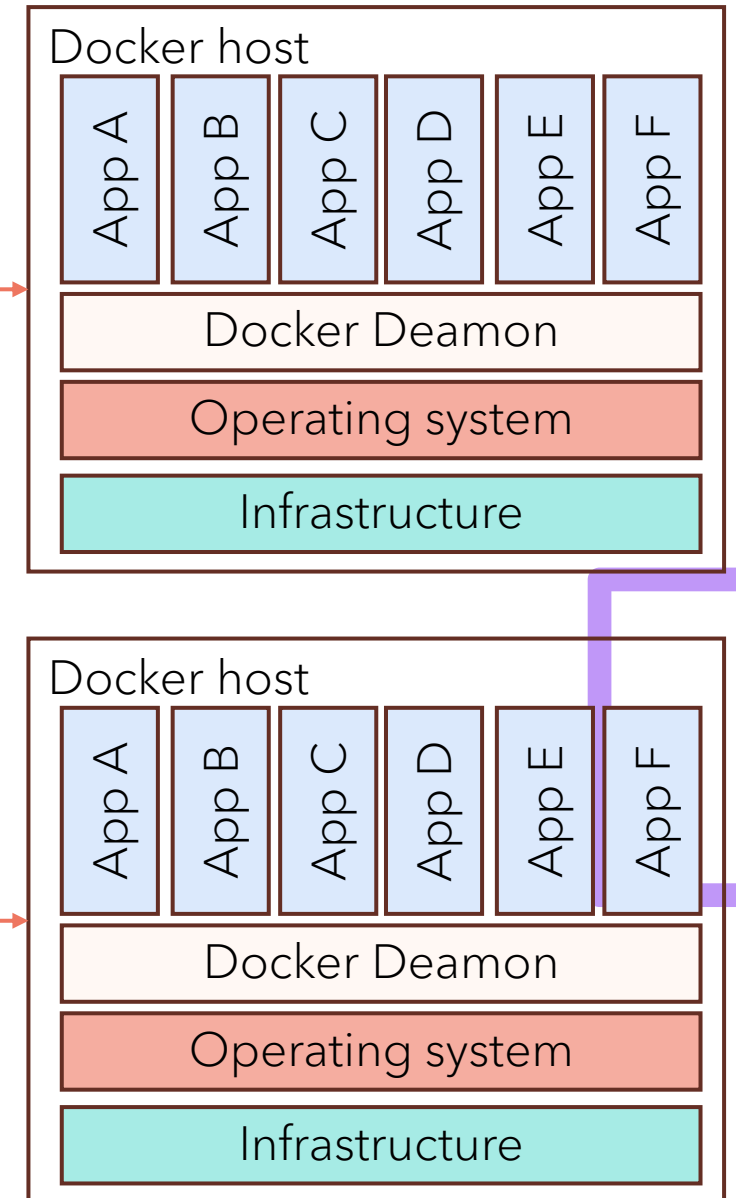
- **Deployment & Scheduling**
Automatically places containers on appropriate servers based on resource needs and constraints.
- **Scaling**
Scales applications up or down depending on demand – automatically adding or removing containers.
- **Load Balancing & Service Discovery**
Automatically routes traffic between containers, ensuring even load distribution.
- **Health Monitoring & Self-Healing**
Detects failed containers and automatically restarts or replaces them.
- **Configuration & Secrets Management**
Manages environment variables, secrets, and configuration across containers securely.
- **Rolling Updates & Rollbacks**
Updates applications seamlessly without downtime, and can revert changes if something breaks.

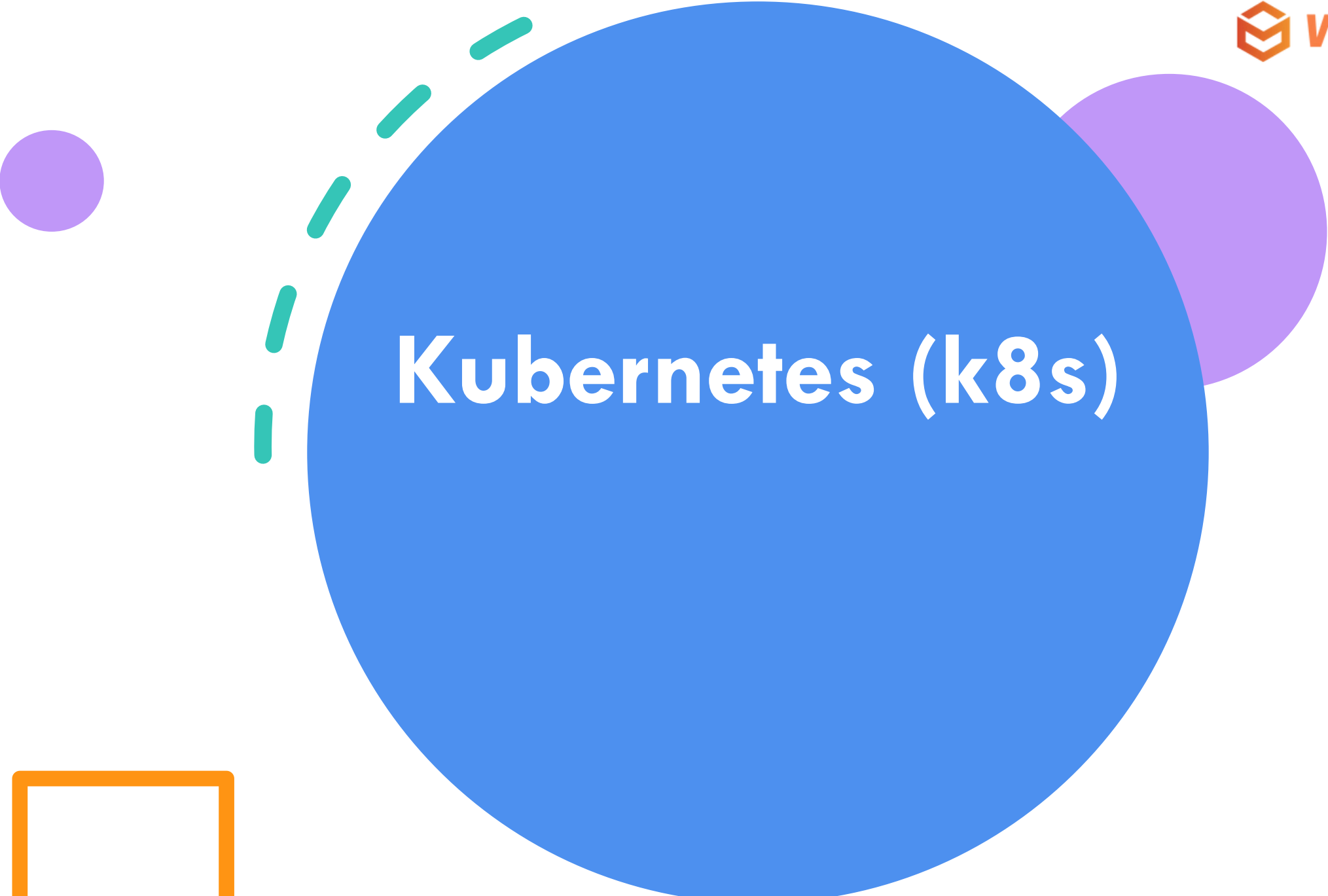


Container Orchestration

Common orchestration platforms include:

- **Kubernetes** (most popular) 
- Docker Swarm
- Red Hat OpenShift (built on Kubernetes)
- Amazon ECS / EKS
- Apache Mesos





Kubernetes (k8s)

What is **Kubernetes (k8s)**?

- Kubernetes (often abbreviated **k8s**) is an open-source container orchestration platform designed to automate the deployment, scaling, and management of containerized applications.
- It was originally developed by Google (based on their internal system, *Borg*) and is now maintained by the **Cloud Native Computing Foundation (CNCF)**.
- The name **k8s** comes from replacing the eight middle letters of *Kubernetes* with "8".



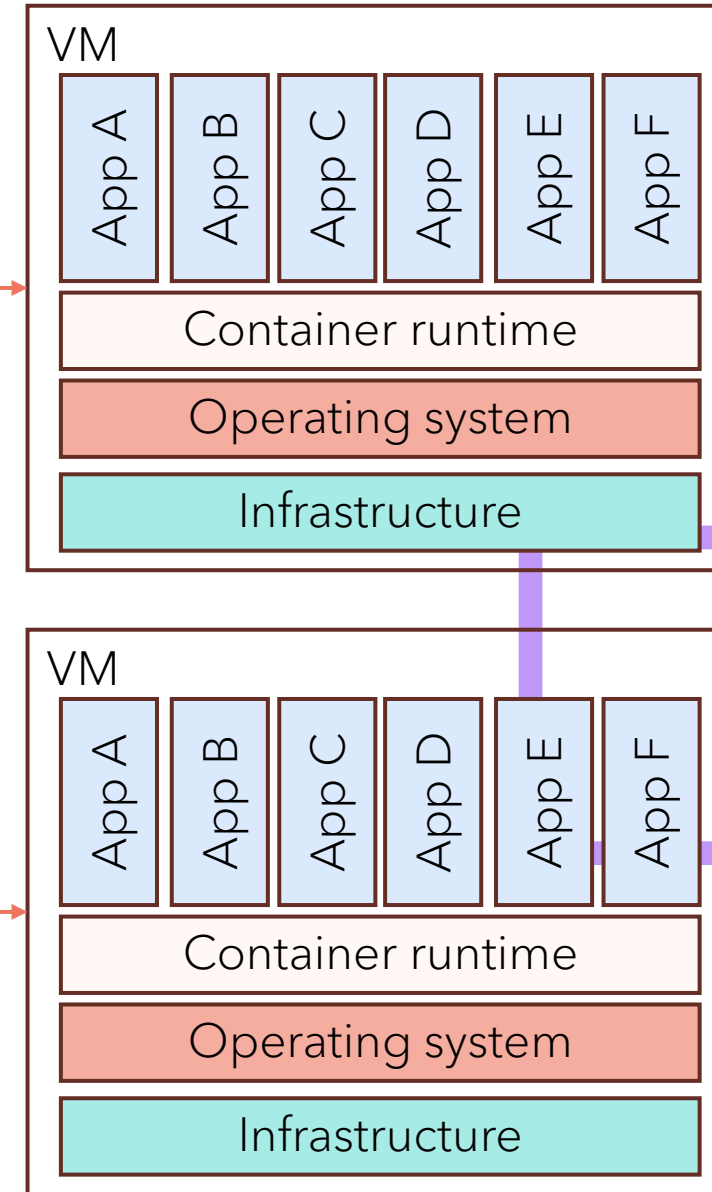
What is Kubernetes (k8s)?

Kubernetes manages **containers** (like Docker containers) across a **cluster** of machines (physical or virtual).

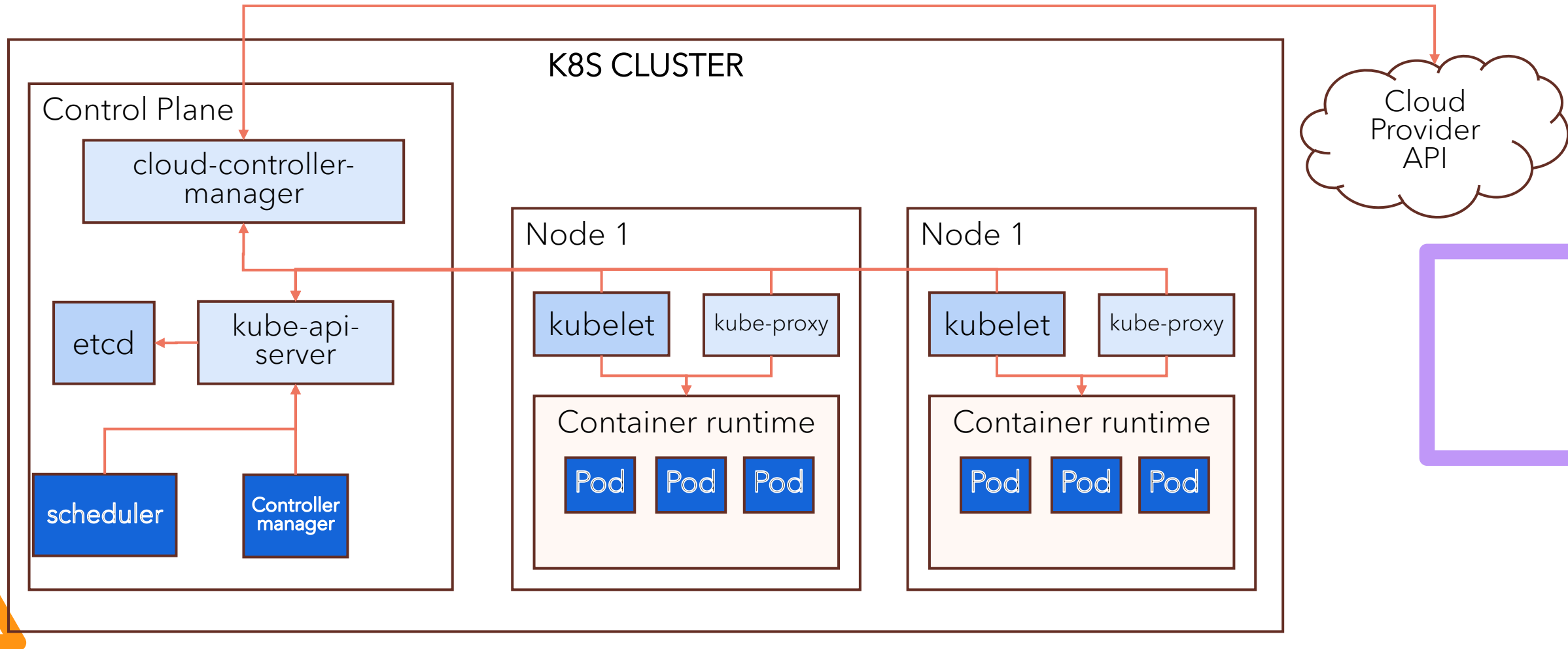
It abstracts those machines into one large compute resource where it:

- **Schedules** containers on suitable machines (based on resource requirements)
- **Manages** networking between containers
- **Monitors** them
- **Restarts** or replaces them automatically if they fail
- **Scales** them up and down based on demand

Essentially, Kubernetes ensures your application is always **running, healthy, and resilient** – without manual intervention.



Kubernetes Architecture



Control Plane Components

The **control plane** manages the overall state of the cluster – making global decisions (like scheduling) and responding to events (like adding new Pods). Usually, all control plane components run on one machine, but in production they can be distributed for high availability.

kube-apiserver

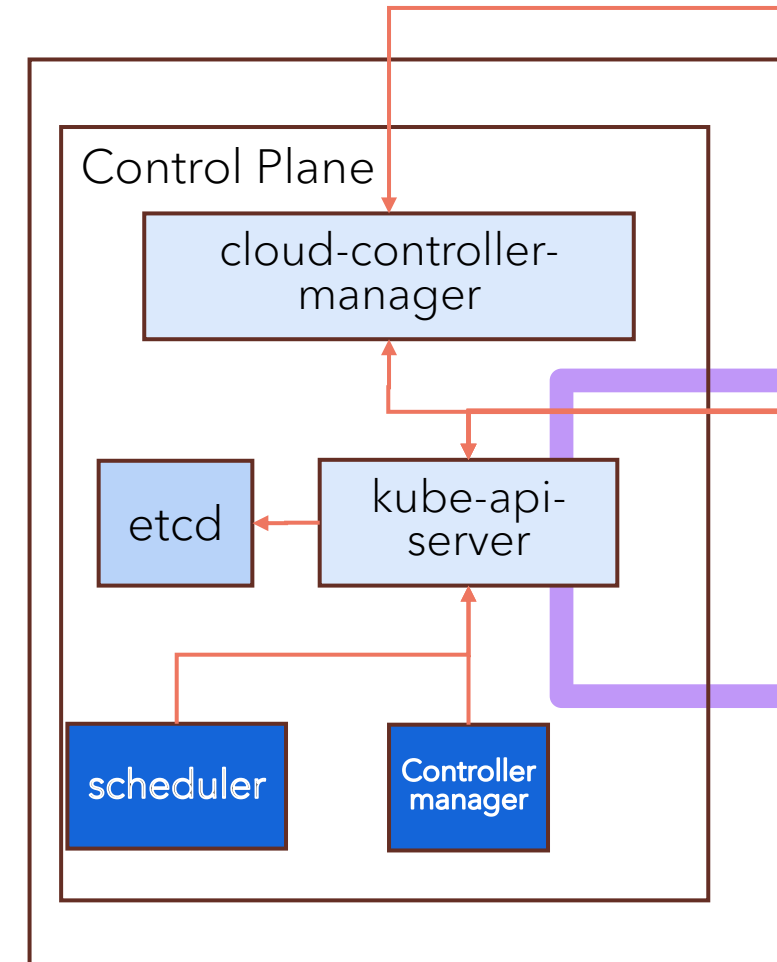
- The **API server** is the front end of Kubernetes, exposing the Kubernetes API. It's scalable and can run multiple instances behind a load balancer.

etcd

- A **highly available key-value store** that holds all cluster data. Back up etcd regularly – it's the single source of truth for the cluster.

kube-scheduler

- Assigns newly created Pods to suitable nodes based on available resources, constraints, affinities, and policies.



Control Plane Components

kube-controller-manager

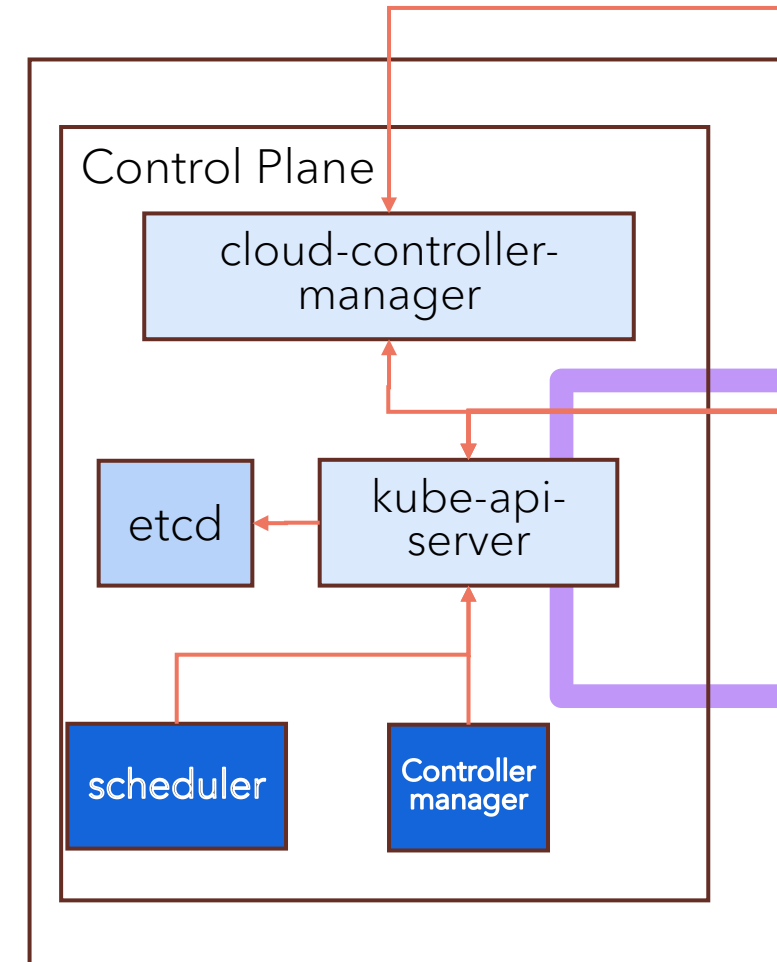
Runs various **controllers** that maintain the desired cluster state.
Examples include:

- **Node controller** – handles node failures
- **Job controller** – manages one-off tasks
- **EndpointSlice controller** – links Services and Pods
- **ServiceAccount controller** – creates default ServiceAccounts

cloud-controller-manager

Handles **cloud provider-specific logic**, such as:

- Checking node status in the cloud
- Managing routes and cloud load balancers
This runs only when integrated with a cloud provider.



Node Components

Run on every node to maintain Pods and provide the container environment.

kubelet

- An **agent** on each node that ensures containers defined in Pod specs are running and healthy.

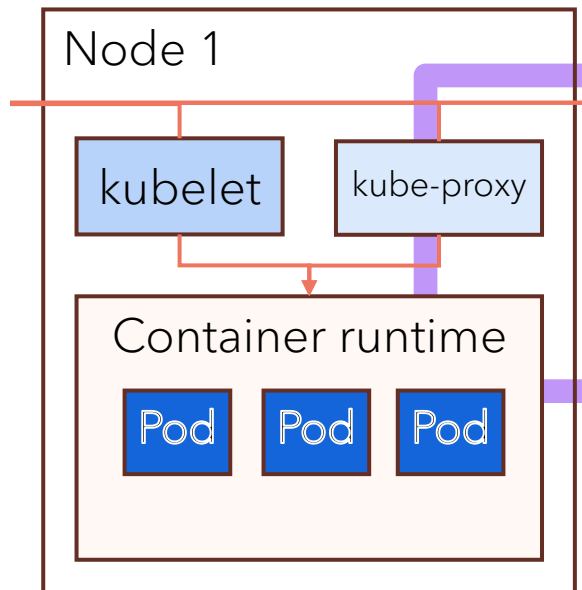
kube-proxy

- A **network proxy** that maintains network rules on nodes, enabling communication between Pods and external services.

Container Runtime

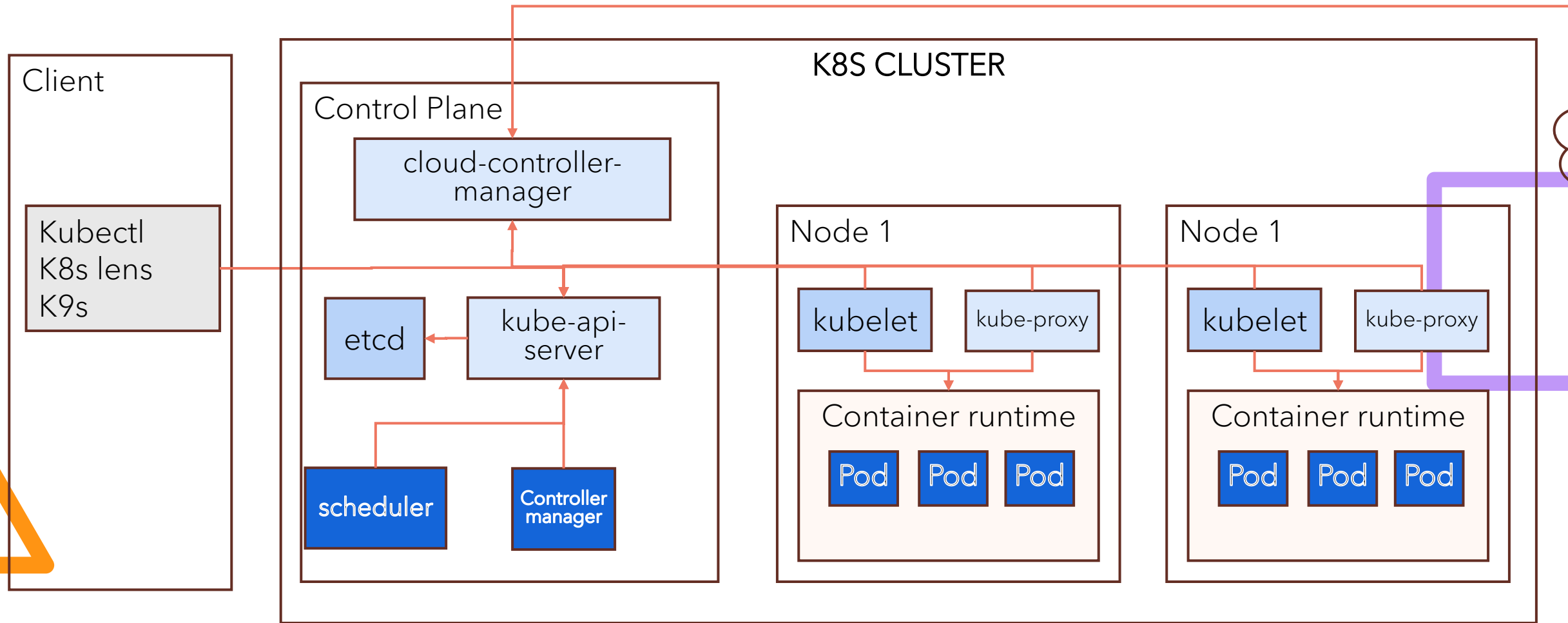
- Software that **runs and manages containers**, such as containerd, CRI-O, or any implementation of the Kubernetes **Container Runtime Interface (CRI)**.

K8S CLUSTER



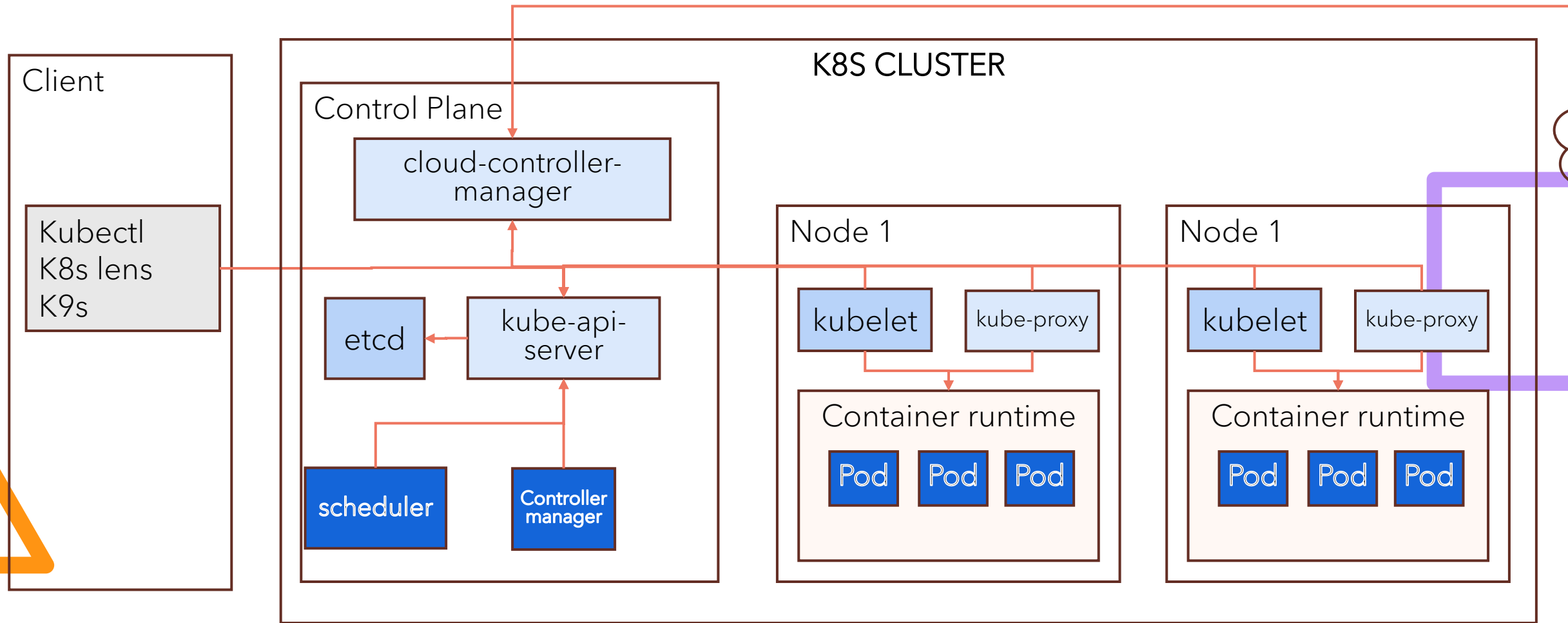
Access to the cluster

- K8s provide rest api to access the cluster
- Client can use **kubectl**, **lens**, **k9s** tools to access api server



Access to the cluster

- K8s provide rest api to access the cluster
- Client can use **kubectl**, **lens**, **k9s** tools to access api server

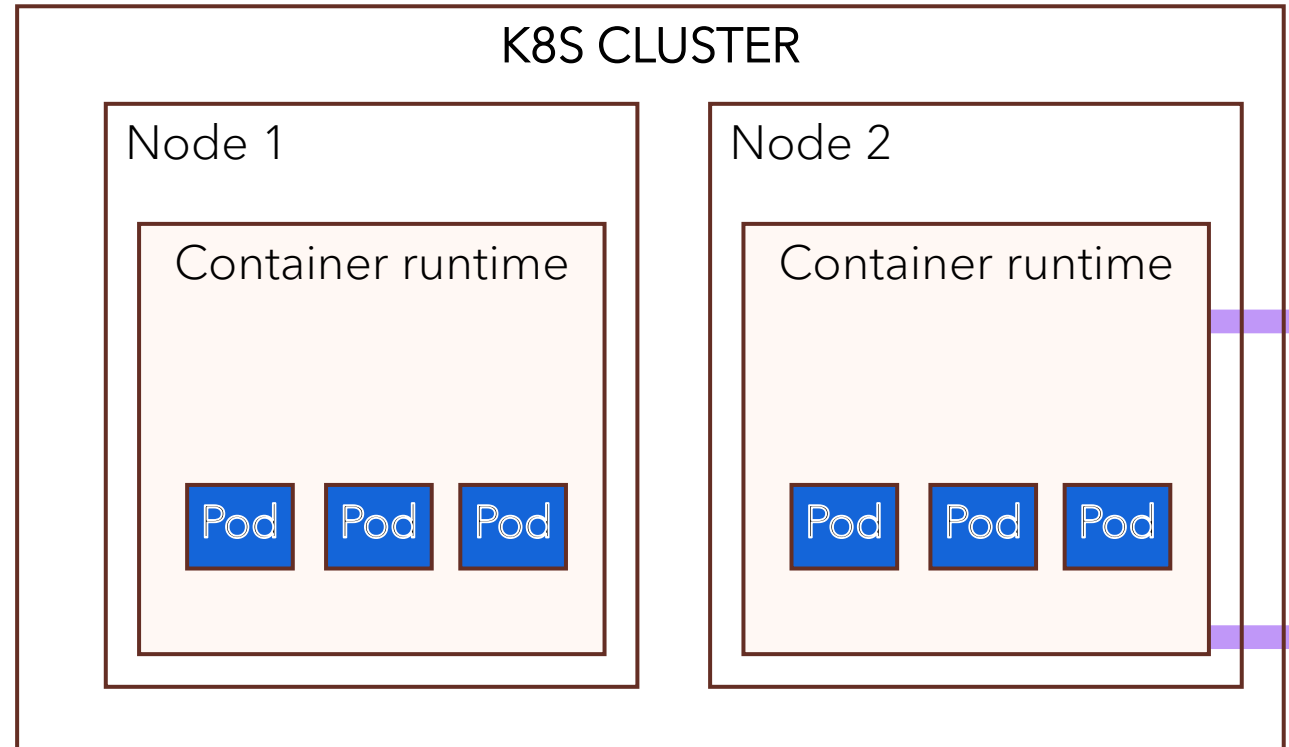




Kubernetes Resources

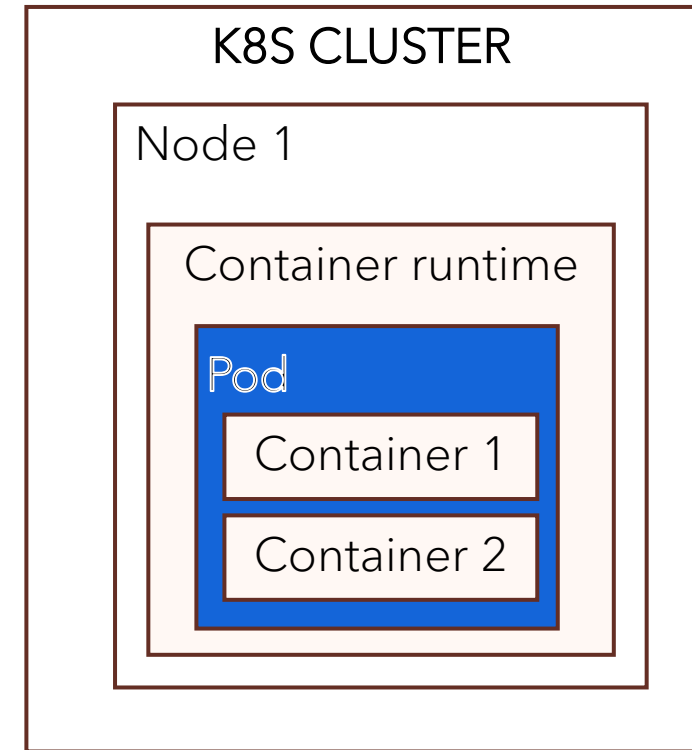
Pod

- A **Pod** is the **smallest deployable unit** in Kubernetes.
- Typically, a Pod runs **a single container**, but multiple containers can run in one Pod if they need to work closely together (e.g., a main app and a helper sidecar).
- If a Pod fails, Kubernetes automatically replaces it with a new one.



Pod

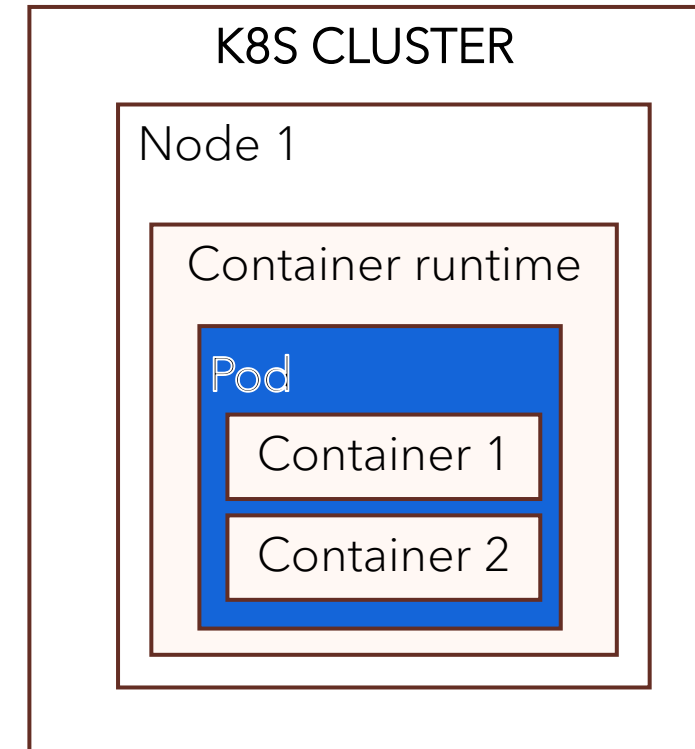
- Inside the pod can have **one or more containers** that share the same:
 - Network (same IP address and port space)
 - Storage volumes
 - Configuration
- Create Pod:
 - `kubectl run my-pod --image=nginx --restart=Never`
 - `kubectl get pods`



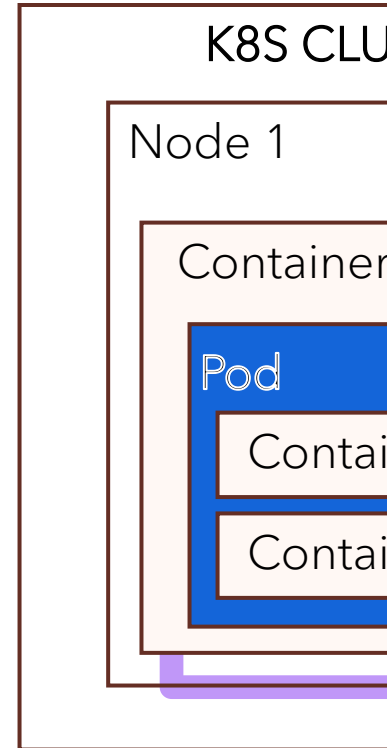
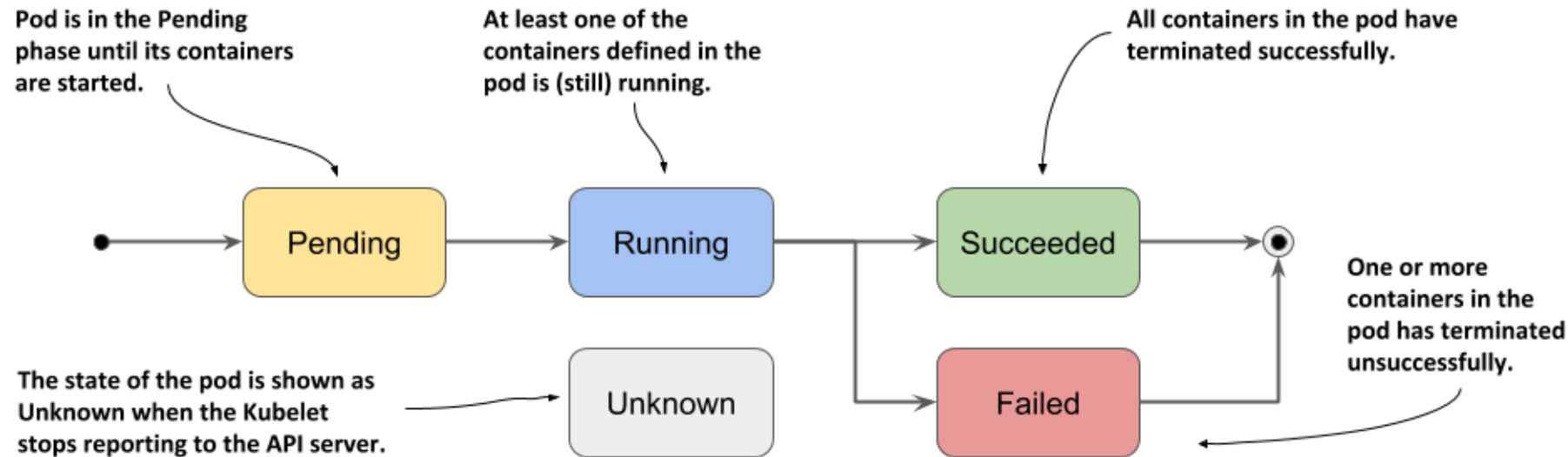
Pod

- Or using YAML
 - `kubectl apply -f pod.yaml`

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

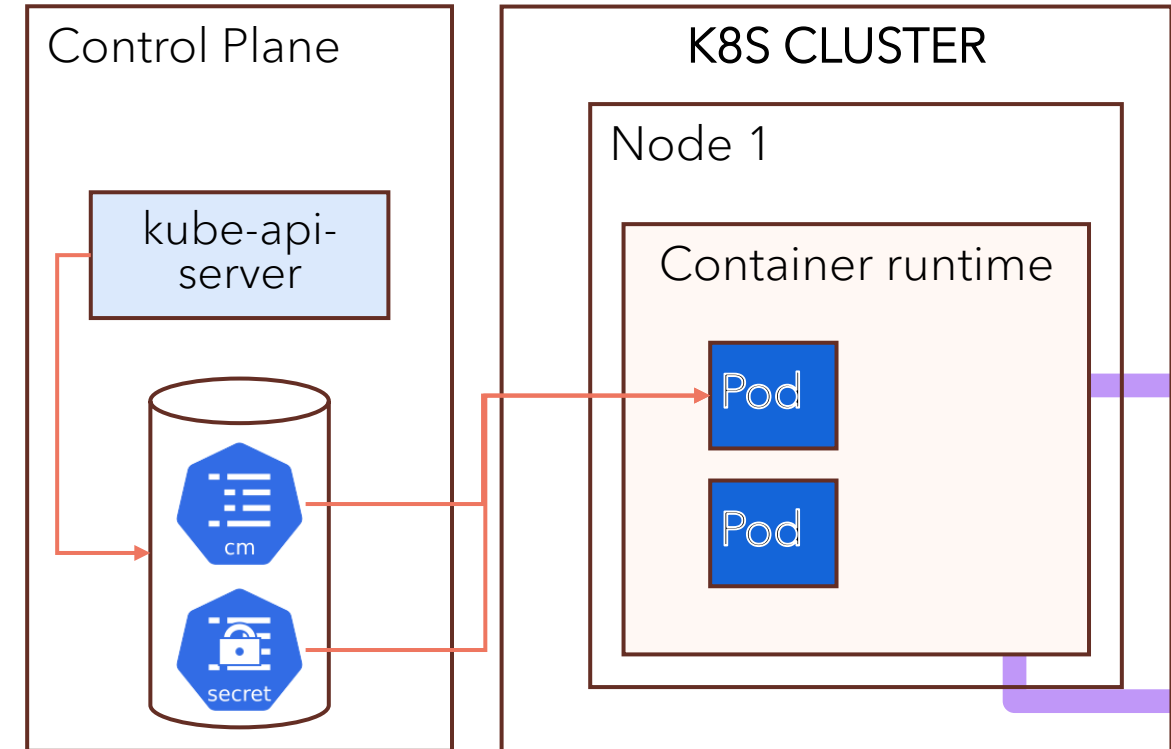


Pod Life Cycle



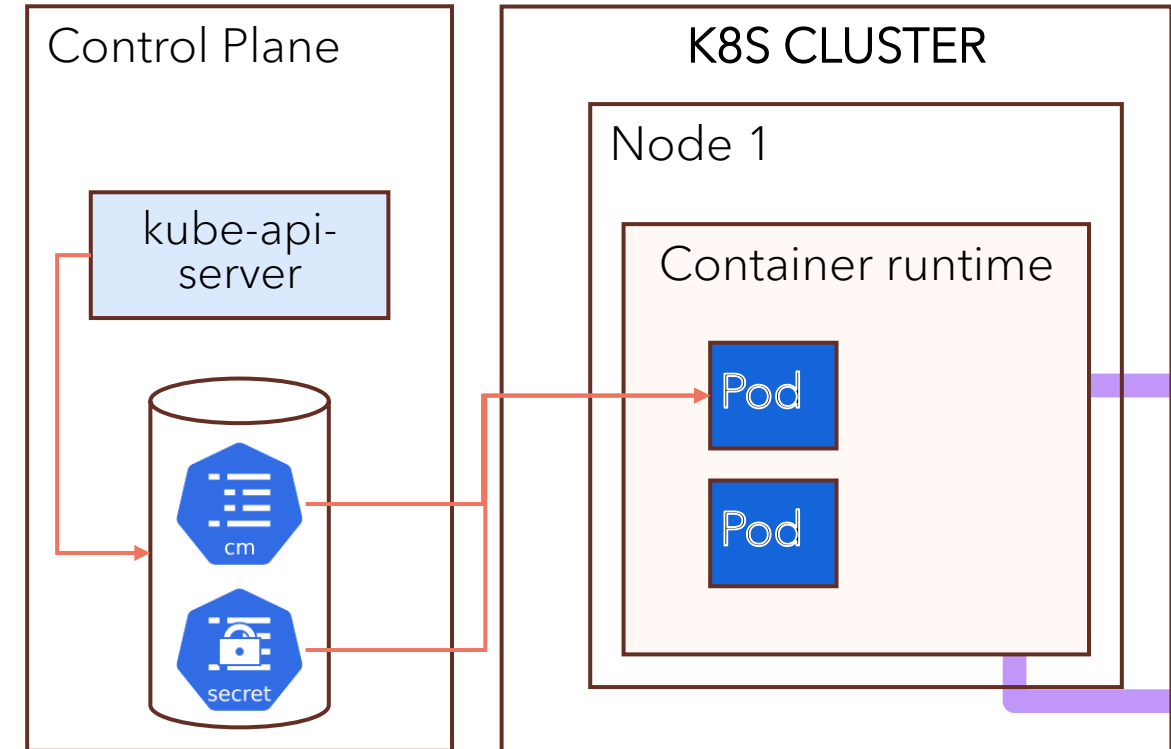
ConfigMap and Secret

- A **ConfigMap** is an API object used to store non-confidential data in key-value pairs. [Pods](#) can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a [volume](#).
- A **Secret** is an object that contains a small amount of sensitive data such as a password, a token, or a key. Such information might otherwise be put in a [Pod](#) specification or in a [container image](#). Using a Secret means that you don't need to include confidential data in your application code.
- Secrets are similar to [ConfigMaps](#) but are specifically intended to hold confidential data.



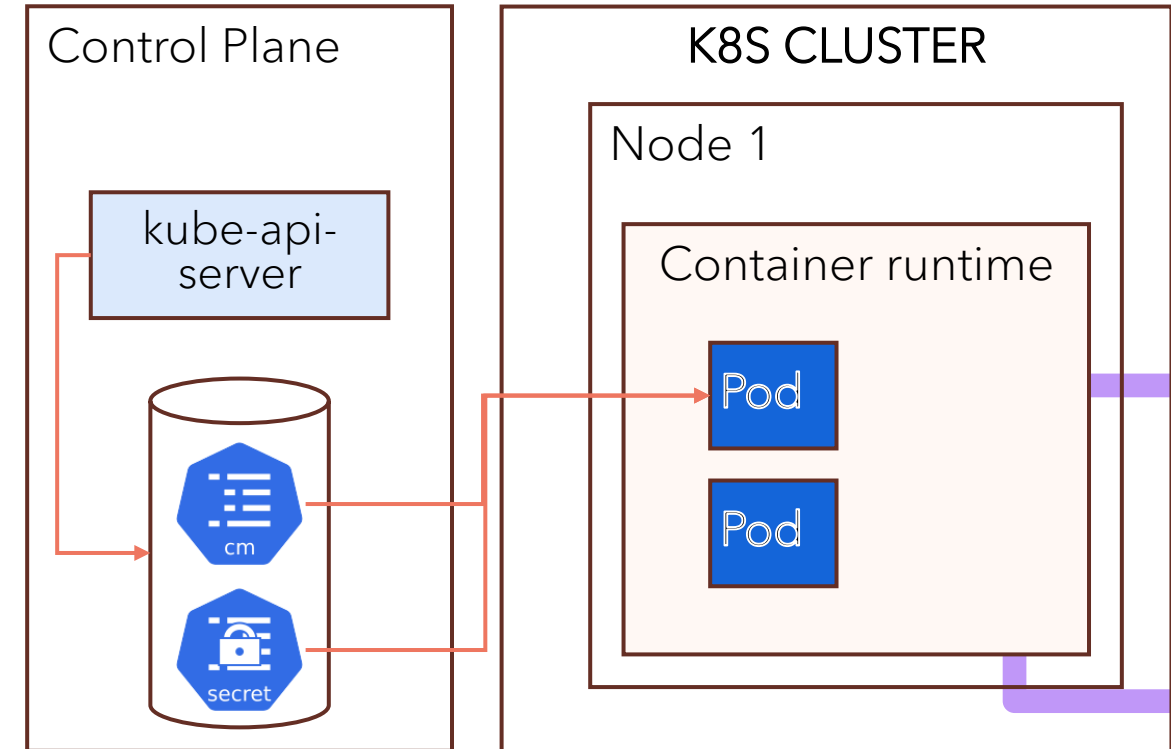
ConfigMap and Secret

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: myconfigmap
data:
  username: k8s-admin
  access_level: "1"
---
apiVersion: v1
kind: Pod
metadata:
  name: env-configmap
spec:
  containers:
    - name: app
      command: ["/bin/sh", "-c", "printenv"]
      image: busybox:latest
      envFrom:
        - configMapRef:
            name: myconfigmap
```



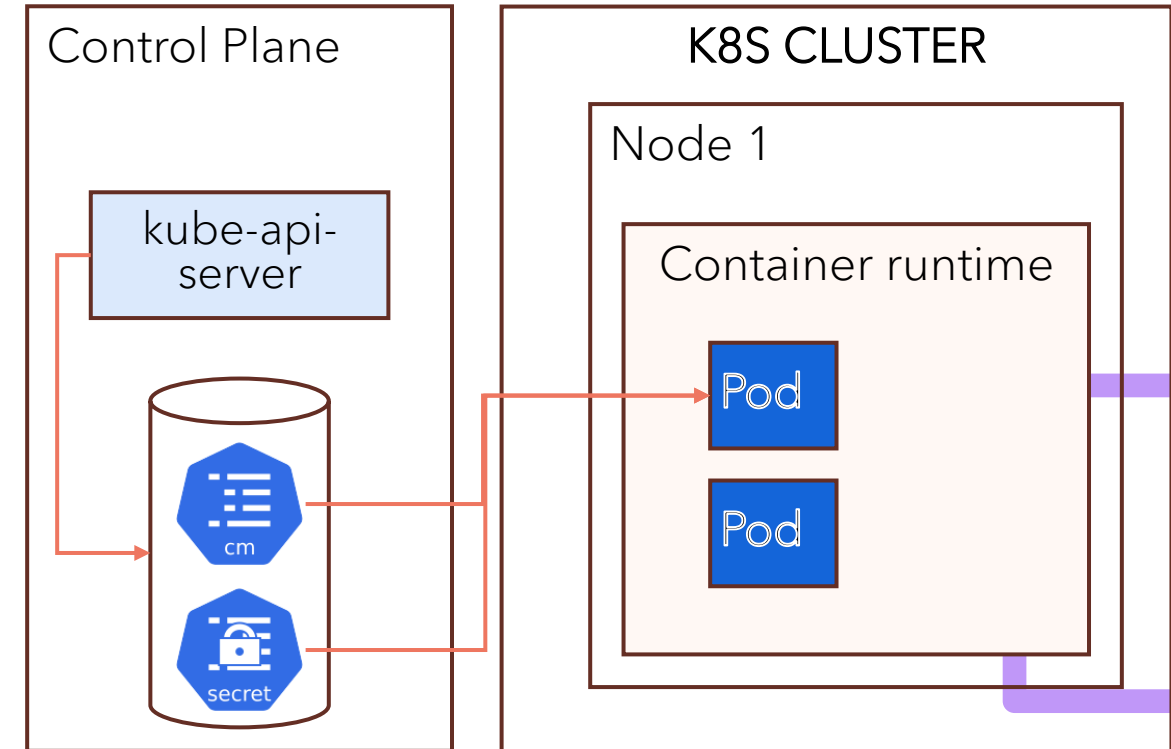
ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: myconfigmap
data:
  username: k8s-admin
  access_level: "1"
---
apiVersion: v1
kind: Pod
metadata:
  name: env-configmap
spec:
  containers:
    - name: app
      command: ["/bin/sh", "-c", "printenv"]
      image: busybox:latest
      envFrom:
        - configMapRef:
            name: myconfigmap
```



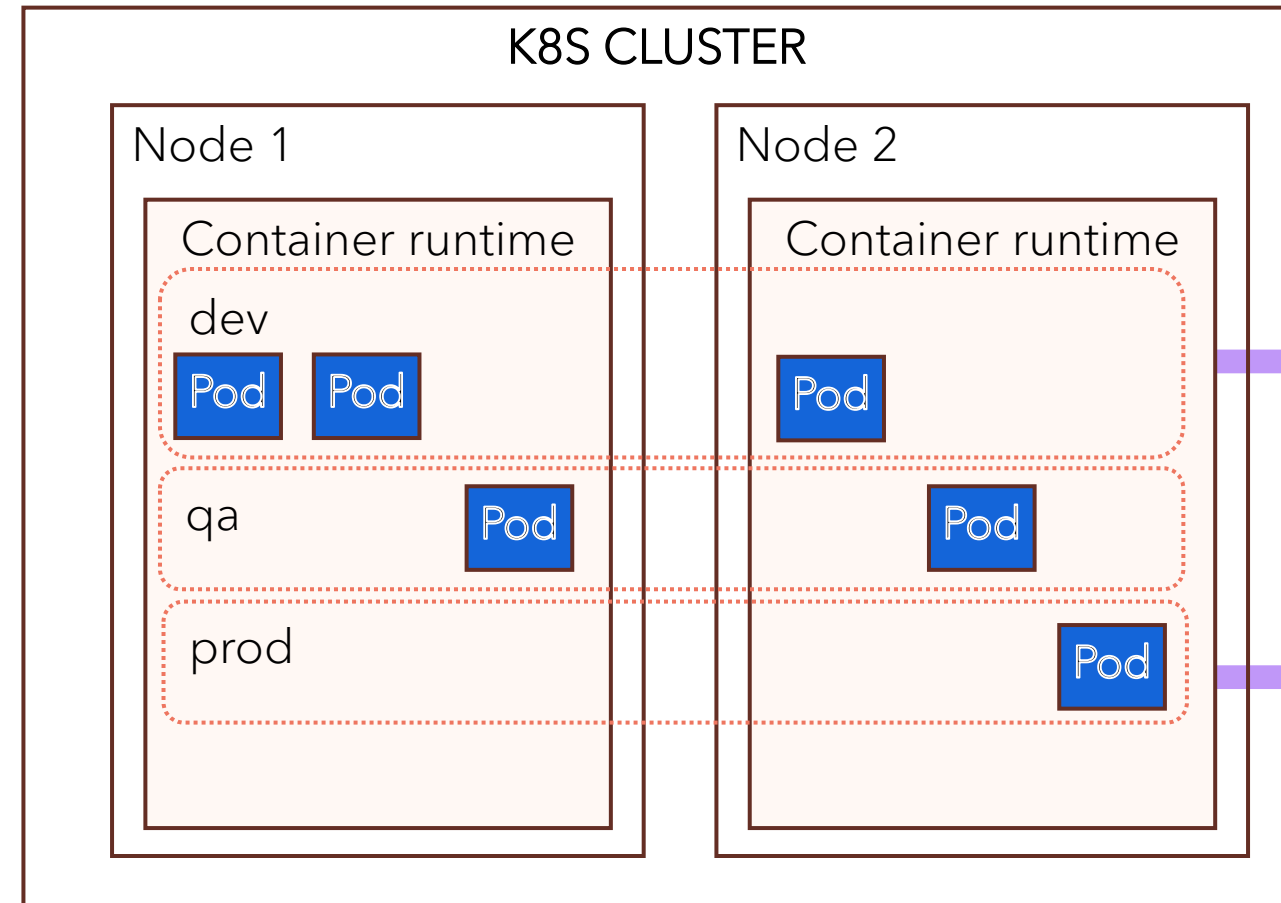
Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
---
apiVersion: v1
kind: Pod
metadata:
  name: secret-dotfiles-pod
spec:
  volumes:
    - name: secret-volume
      secret:
        secretName: dotfile-secret
  containers:
    - name: dotfile-test-container
      image: registry.k8s.io/busybox
      command:
        - ls
        - "-l"
        - "/etc/secret-volume"
      volumeMounts:
        - name: secret-volume
          readOnly: true
          mountPath: "/etc/secret-volume"
```



Namespace

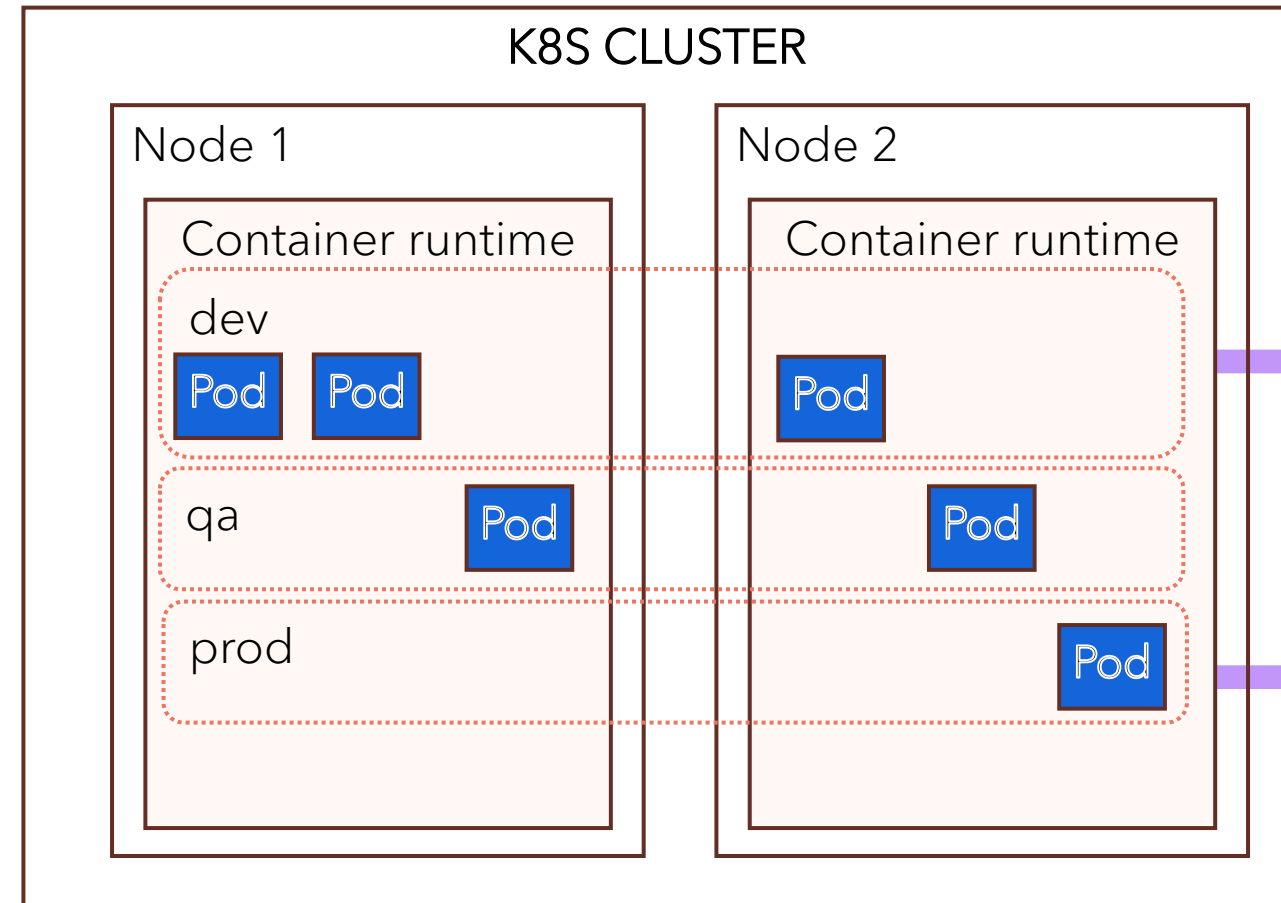
- A **Namespace** is a **logical partition** within a Kubernetes cluster.
- It helps organize and isolate resources (Pods, Services, Deployments, etc.) into separate environments.
- Useful for:
 - Multi-team or multi-project clusters
 - Environment separation (e.g., dev, test, prod)
 - Applying resource quotas or access control



Namespace

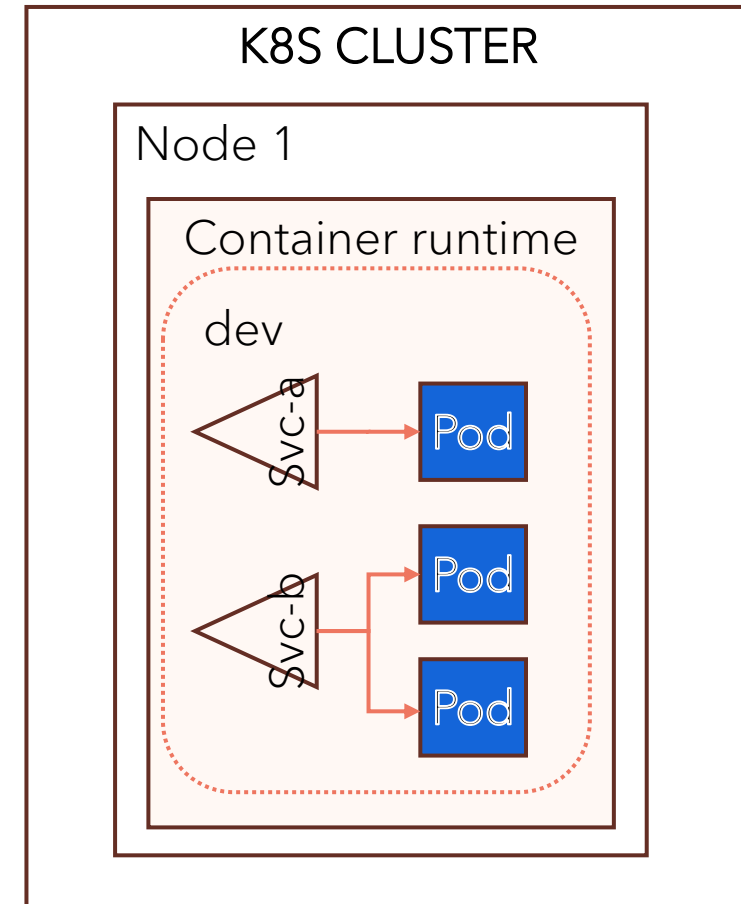
- Create namespace
 - `kubectl create namespace my-namespace`
- Or using yaml
 - `kubectl apply -f ns.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: my-namespace
```



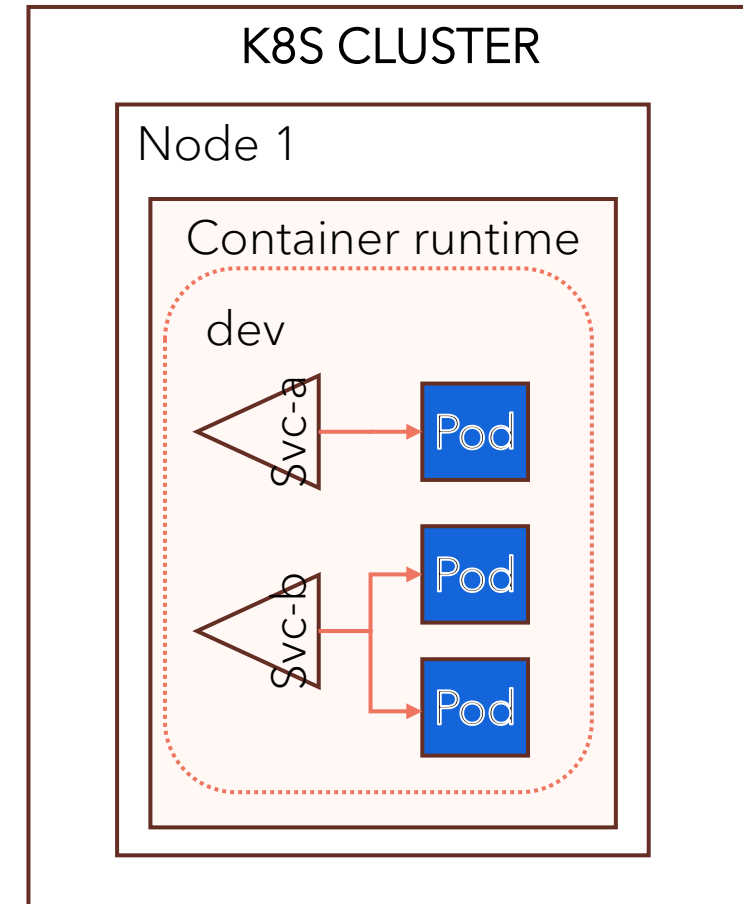
Service

- A **Service** provides a **stable network endpoint** to access one or more Pods.
- Since Pods are ephemeral (they can be replaced and get new IPs), a Service gives a **consistent way** to communicate with them.
- It performs **load balancing** among Pods that match its selector.



Service

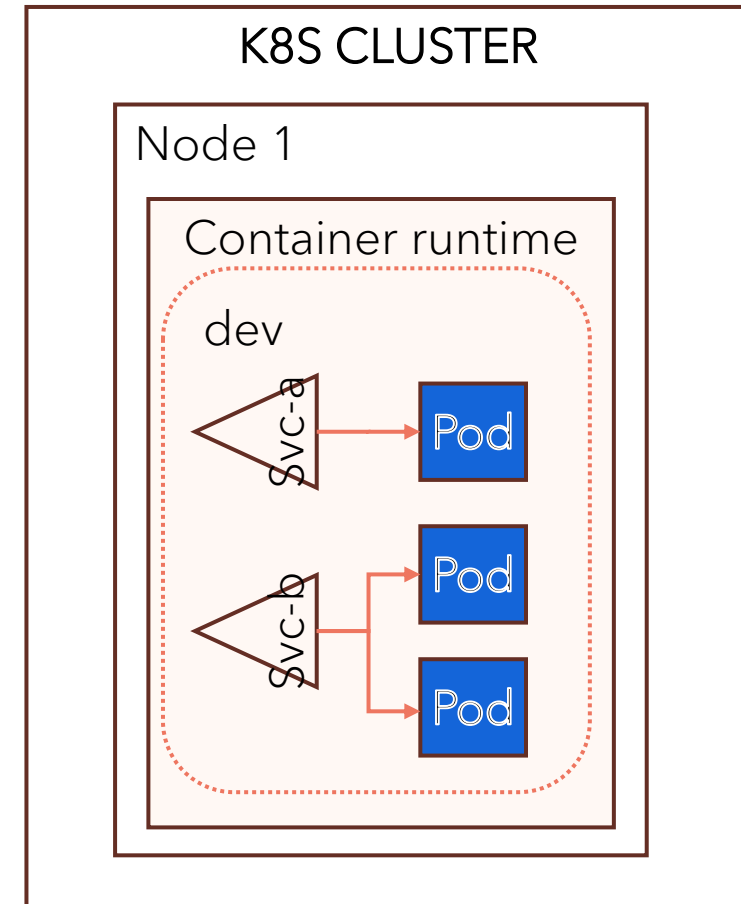
Type	Description
ClusterIP	Default; accessible only inside the cluster
NodePort	Exposes the Service on each node's IP at a static port
LoadBalancer	Creates an external load balancer (used in cloud environments)
ExternalName	Maps a Service to an external DNS name



Service

- Create namespace
 - `kubectl expose pod nginx-pod --port=80 --target-port=80 --name=nginx-service`
- Or using yaml
 - `kubectl apply -f svc.yaml`

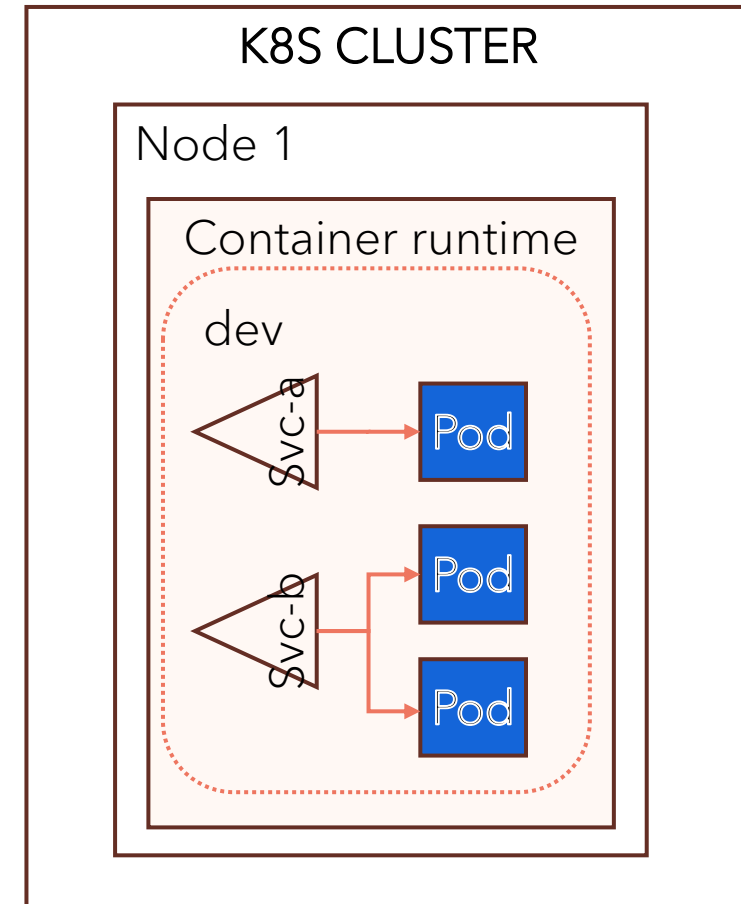
```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx          # Must match labels on your Pod(s)
  ports:
    - port: 80          # Port exposed by the Service
      targetPort: 80     # Port the container listens on
  type: ClusterIP       # Can be ClusterIP, NodePort, LoadBalancer, etc.
```



Service

- NodePort

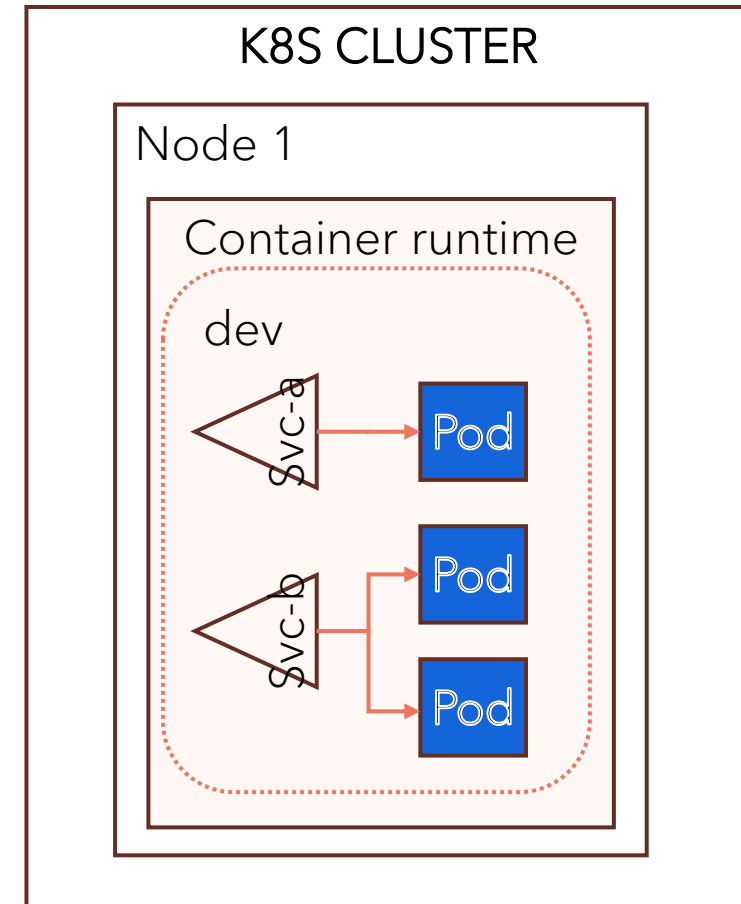
```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```



Service

- Load Balancer

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

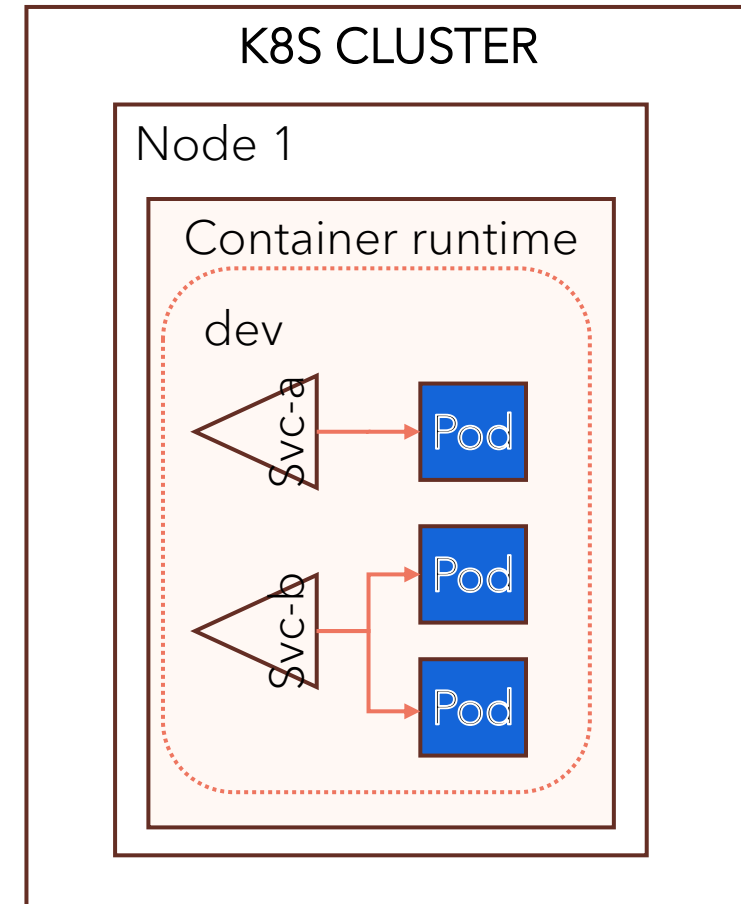


DNS resolution

Kubernetes leverages DNS for service discovery and inter-pod communication within a cluster. This system translates human-readable service names into machine-readable IP addresses, simplifying network interactions for applications running in Kubernetes.

DNS Records for Services and Pods:

- Service:
 - `<svc-name>.namespace`
 - `<svc-name>.namespace.svc.cluster.local`
- Pods:
 - `pod-ip-address.namespace.pod.cluster.local`

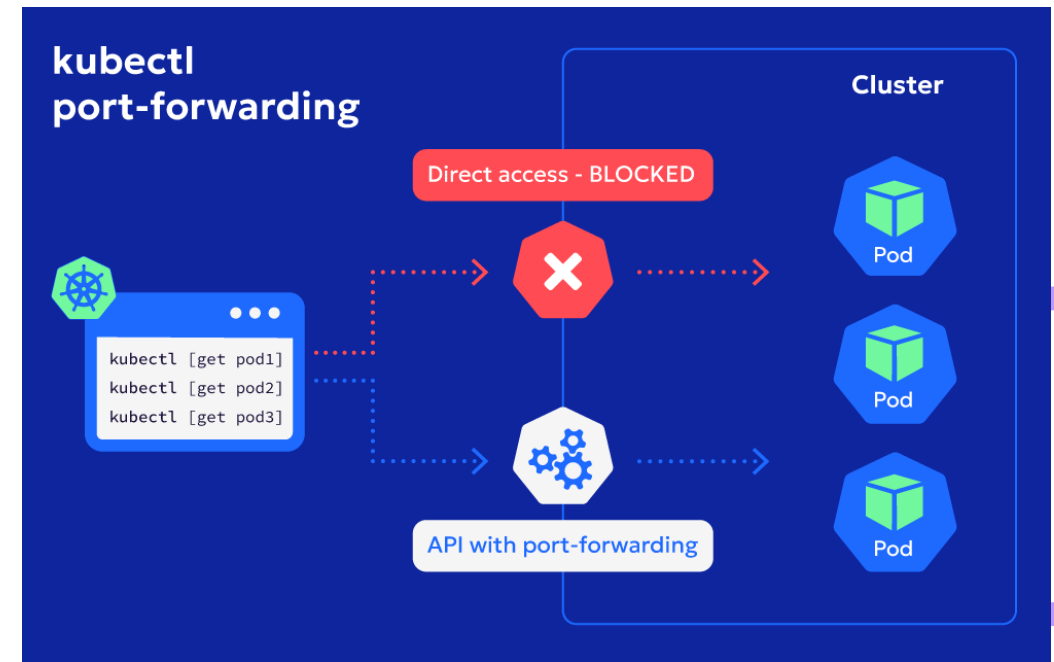


Port forwarding

Sometimes you need to access something inside your cluster that's not exposed via a LoadBalancer or Ingress. Maybe you're testing, debugging, or checking logs, but you don't want to (or shouldn't) expose the service externally.

Common use cases:

- Debugging internal microservices.
- Connecting to in-cluster databases.
- Viewing internal dashboards (like Prometheus or Grafana).
- Running local tests on services without exposing them.



Port forwarding

Forward a local port to a pod:

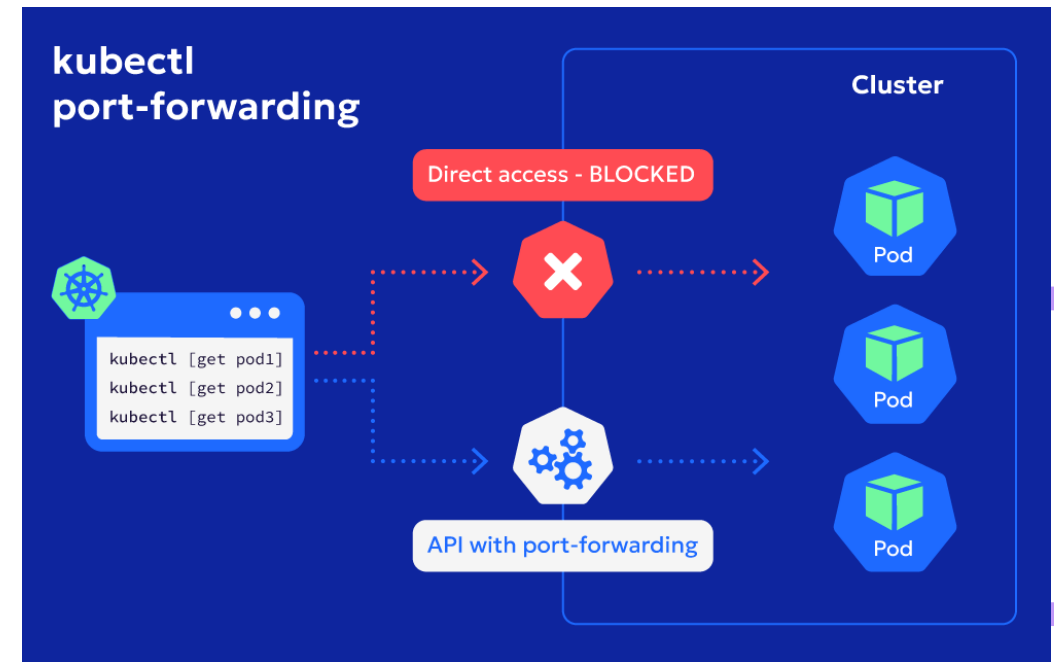
- `kubectl port-forward pod/my-pod-name 8080:80`

This forwards `localhost:8080` on your machine to port `80` on `my-pod-name`.

Forward a local port to a service:

- `kubectl port-forward svc/my-service 5432:5432`

Now you can connect to `localhost:5432`, and traffic goes to your Kubernetes service on port `5432`.





Practice Lab